

DD: BIL-01 v1.1 — Charging Engine

Developer implementation guide: DD_BIL-01-Charging_Engine-Core-DEV_IMPL_GUIDE-v1.1.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_BIL-01-Charging_Engine-Core-v1.1.md.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Billing | **Version:** v1.1 The charging engine. Reads BIL-CFG-01 catalog at runtime to resolve which BillableEvents fire for a given workflow intent or lifecycle trigger; computes amounts (signed where applicable); aggregates line items into a canonical envelope; settles per billing mode (PREPAID wallet debit vs POSTPAID invoice line attachment); invokes optional SUB-LM-01 state callbacks; emits domain events to Kafka. Called synchronously by subscription workflows via the bil01-emit-intent worker. Called asynchronously via Kafka consumers for LIFECYCLE_EVENT triggers. Called directly via REST for ADMIN_ACTION / CUSTOMER_PURCHASE / EXTERNAL_PAYMENT. **What changed from v0.9:** - **Formal bil01-emit-intent worker contract** — canonical request envelope, canonical response shape with lineItems[] and totals (signed). Workflow DDs cite this contract. - **Signed-amount runtime computation** — implementation of amount_sign_policy = SIGNED from BIL-CFG-01 v1.0. The billing-rules KJAR computes a signed delta for events like MIGRATION_PRORATE_DELTA where direction depends on source vs target Package price. - cycleAnchorPolicy **interpretation** — for UPGRADE/DOWNGRADE/MIGRATION intents: PRESERVE returns proration delta (signed); RESET_TO_OPERATION_DATE returns OLD_CYCLE_CREDIT + FIRST_CYCLE_CHARGE. Zero-delta short-circuit for effectiveTiming = END_OF_CURRENT_CYCLE + PRESERVE. - **Operator scoping** — every call carries operatorCode; catalog resolution filters by (operatorCode, triggerType, triggerIntentCode) using BIL-CFG-01 v1.0's operator-scoped indexes. - **Section cleanup** — removed Module placement, Codebase, Foundation-specific configurations, Frontend, Gap & bridge, Migration approach, What's NEW in V3 per memory rule #26.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-01-Charging_Engine-Core-v1.1.md
Version	v1.1
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/billing/decisions/bil_dec_01HZ_OTIENO_MIGRATE
- GET /api/billing/decisions/{decisionId}
- POST /api/billing/charge-direct
- POST /api/billing/decide
- POST /api/invoice-lines
- POST /api/wallet-debit

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- payment-confirmed

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Billing | **Version:** v1.1

The charging engine. Reads BIL-CFG-01 catalog at runtime to resolve which BillableEvents fire for a given workflow intent or lifecycle trigger; computes amounts (signed where applicable); aggregates line items into a canonical envelope; settles per billing mode (PREPAID wallet debit vs POSTPAID invoice line attachment); invokes optional SUB-LM-01 state callbacks; emits domain events to Kafka.

Called synchronously by subscription workflows via the `bil01-emit-intent` worker. Called asynchronously via Kafka consumers for `LIFECYCLE_EVENT` triggers. Called directly via REST for `ADMIN_ACTION` / `CUSTOMER_PURCHASE` / `EXTERNAL_PAYMENT`.

What changed from v0.9:

- **Formal bil01-emit-intent worker contract** — canonical request envelope, canonical response shape with `lineItems[]` and `totals` (signed). Workflow DDs cite this contract.
- **Signed-amount runtime computation** — implementation of `amount_sign_policy = SIGNED` from BIL-CFG-01 v1.0. The billing-rules KJAR computes a signed delta for events like `MIGRATION_PRORATE_DELTA` where direction depends on source vs target Package price.
- **cycleAnchorPolicy interpretation** — for `UPGRADE/DOWNGRADE/MIGRATION` intents: `PRESERVE` returns proration delta (signed); `RESET_TO_OPERATION_DATE` returns `OLD_CYCLE_CREDIT + FIRST_CYCLE_CHARGE`. Zero-delta short-circuit for `effectiveTiming = END_OF_CURRENT_CYCLE + PRESERVE`.
- **Operator scoping** — every call carries `operatorCode`; catalog resolution filters by (`operatorCode`, `triggerType`, `triggerIntentCode`) using BIL-CFG-01 v1.0's operator-scoped indexes.
- **Section cleanup** — removed `Module placement`, `Codebase`, `Foundation-specific configurations`, `Frontend`, `Gap & bridge`, `Migration approach`, What's NEW in V3 per memory rule #26.

Glossary

Term	Meaning
Charging engine	This DD. Reads BIL-CFG-01 catalog, resolves matching BillableEvents, computes amounts, aggregates line items, settles, invokes state callbacks.
Intent	A canonical workflow operation. Carried by <code>bil01-emit-intent</code> calls. Examples: <code>ACTIVATION_INTENT</code> , <code>PAUSE_INTENT</code> , <code>UPGRADE_INTENT</code> , <code>MIGRATION_INTENT</code> .
Intent context	The workflow-supplied facts that drive filter rule evaluation and price computation. Example fields: <code>cycleAnchorPolicy</code> , <code>effectiveTiming</code> , <code>equipmentPath</code> , <code>sameFranchise</code> , <code>sourcePackagePrice</code> , <code>targetPackagePrice</code> , <code>cycleWindow</code> . Carried in the canonical envelope.

Term	Meaning
Canonical envelope	The request/response shape used by bil01-emit-intent. Request carries operatorCode, subscriptionId, customerId, intentCode, intentContext, fulfillmentCorrelationId. Response carries decisionId, lineItems[], totals (signed), payFirstRequired, stateCallbackInvocations[].
Line item	One row in the response, derived from one BillableEvent match. Carries billableEventCode, billableEventCategory, amount (signed per amount_sign_policy), currency, description, metadata.
Totals	The aggregated {gross, tax, net} across all line items. Signed — may be positive (net charge), zero, or negative (net credit). For POSTPAID, negative net is rejected per R-SE-3.
Settlement	Post-aggregation step: PREPAID → one wallet debit of the signed total; POSTPAID → invoice line attachment via BIL-02-GEN-01.
Pay-first gate	A workflow waits for BIL-01 settlement success before proceeding. Required when any matched event has pay_first_required = true.
State callback	An invocation of SUB-LM-01 after successful settlement, per the matched BillableEvent's state_callback.
Filter rule	A Drools rule in the operator-scoped billing-rules-<operator> KJAR. Receives the intent context as facts and votes accept/reject for a candidate BillableEvent.

1. What this DD owns

1. **The bil01-emit-intent worker contract** — canonical request/response envelope.
2. **Catalog resolution** — for a given intent (or lifecycle event, or direct admin action), resolve matching BillableEvents from BIL-CFG-01 filtered by operator scope + trigger metadata + filter rules.
3. **Amount computation** — formulas for proration, ETF, fixed fees, signed deltas. Runtime sign resolution per amount_sign_policy.
4. **cycleAnchorPolicy + effectiveTiming interpretation** — for package-change intents.
5. **Line item aggregation** — multi-event invocations produce a single envelope.
6. **Settlement** — PREPAID wallet path + POSTPAID invoice path.
7. **State callback invocation** — post-settlement SUB-LM-01 calls per matched event's state_callback.
8. **Domain Kafka events** — BillableEventCharged, BillableEventStateCallbackFailed.

What this DD does NOT own

- **BillableEvent catalog** — BIL-CFG-01 v1.0.
- **Service catalog** — PLM-CFG-01 v1.0.
- **Package + Package Version catalog** — SIP-01.
- **Tax computation** — PLM-CFG-02 (BIL-01 applies but doesn't define).
- **Wallet operations** — PLM-CFG-03 + BIL-01-PAY-01.
- **Invoice generation** — BIL-02-GEN-01.
- **Credit note flow** — BIL-02-ADJ-01.
- **Collections / dunning** — BIL-04.
- **Subscription state transitions** — SUB-LM-01 (BIL-01 invokes callbacks, doesn't own transitions).
- **Cycle accrual for CONTINUOUS Packages** — BIL-02-GEN-01 (recurring billing). BIL-01 handles one-shot events only.

2. Canonical envelope

2.1 Request

The bil01-emit-intent worker (or direct REST caller) sends:

```

{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_OTIENO_FIBER",
  "customerId": "cust_01HZ_OTIENO",
  "intentCode": "MIGRATION_INTENT",
  "intentContext": {
    "sourceHomepassId": "hp_01HZ_KAREN_017",
    "sourceFranchiseId": "frn_NRB_KAREN_RES",
    "sourceNetworkRootType": "GPON",
    "sourcePackageRef": "pkg_01HX_INET_100M_VOIP_RES",
    "sourcePackageVersionId": "pkv_01HX_INET_100M_V3",
    "sourcePackagePrice": 2500.00,
    "targetHomepassId": "hp_01HZ_LAVI_073",
    "targetFranchiseId": "frn_NRB_LAVI_RES",
    "targetNetworkRootType": "GPON",
    "sameFranchise": false,
    "targetPackageRef": "pkg_01HX_INET_VOIP_IPTV_RES",
    "targetPackageVersionId": "pkv_01HX_INET_VOIP_IPTV_V1",
    "targetPackagePrice": 7500.00,
    "currency": "KES",
    "equipmentPath": "NEW_INSTALL",
    "disconnectMode": "TECH_DISPATCH",
    "migrationTrigger": "CUSTOMER_REQUESTED_MIGRATION",
    "cycleAnchorPolicy": "PRESERVE",
    "effectiveTiming": "SCHEDULED_AT",
    "scheduledCommitAt": "2026-06-01T09:00:00+03:00",
    "cycleWindow": {"startedAt": "2026-05-01T00:00:00+03:00", "endsAt": "2026-06-01T00:00:00+03:00"}
  },
  "fulfillmentCorrelationId": "subop_01HZ_OTIENO_MIGRATE:bil01",
  "callerWorkflowKind": "SUB_WF_MIGRATION",
  "callerWorkflowRefId": "subop_01HZ_OTIENO_MIGRATE"
}

```

intentContext fields are intent-specific. The fields above are common to package-change + HomePass-change intents. For ACTIVATION_INTENT the context carries packageRef, packageVersionId, cycleAnchorOverflowApplied, etc. For PAUSE_INTENT it carries pauseReasonCode, pauseDurationDays. For TERMINATION_INTENT it carries coolingOffApplicable, contractMinimumTermMonths, daysActiveInContract, prepaidWalletBalance. Filter rules in the billing-rules KJAR read whichever fields they need from this open-shaped object.

2.2 Response

```

{
  "decisionId": "bil_dec_01HZ_OTIENO_MIGRATE",
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_OTIENO_FIBER",
  "intentCode": "MIGRATION_INTENT",
  "fulfillmentCorrelationId": "subop_01HZ_OTIENO_MIGRATE:bil01",
  "lineItems": [
    {
      "lineItemId": "bil_line_01HZ_OTIENO_MIGRATE_FEE",
      "billableEventCode": "MIGRATION_FEE",
      "billableEventCategory": "LIFECYCLE_FEE",
      "amount": 1500.00,
      "currency": "KES",
      "description": "Standard customer-initiated migration fee",
      "metadata": {"sameFranchise": false}
    },
    {
      "lineItemId": "bil_line_01HZ_OTIENO_MIGRATE_PRORATE",
      "billableEventCode": "MIGRATION_PRORATE_DELTA",
      "billableEventCategory": "PRORATION",
      "amount": 5000.00,
      "currency": "KES",
      "description": "Migration proration delta – 30 days remaining at price difference",
      "metadata": {"daysRemaining": 30, "priceDelta": 5000.00, "computedSign": "POSITIVE"}
    }
  ],
  "totals": {
    "gross": 6500.00,
    "tax": 1040.00,
    "net": 7540.00,
    "currency": "KES"
  },
  "payFirstRequired": true,
  "settlementStatus": "PENDING",
  "stateCallbackInvocations": [],
  "decidedAt": "2026-06-01T09:00:02+03:00"
}

```

The workflow worker reads `payFirstRequired` and `settlementStatus` to know whether to park on payment-confirmed before proceeding. The worker reads `totals.net` to display the customer-facing charge. The worker reads `lineItems[]` to render the customer's bill preview.

When `settlementStatus` is initially PENDING, BIL-01 immediately proceeds to settle (PREPAID wallet debit or POSTPAID invoice line attachment). The settlement outcome is reflected via the operational `BillableEventCharged` Kafka event (R-EV-1) and via a payment-confirmed message to the workflow's BPMN.

If `totals.net <= 0` for POSTPAID, the response carries `settlementStatus = REJECTED` with `reasonCode = REQUIRES_CREDIT_NOTE_WORKFLOW` per R-SE-3.

3. Rules

3.1 Catalog resolution

Rule	Description
R-BIL-01-CR-1	For SAGA_INTENT calls, BIL-01 queries BIL-CFG-01 with (<code>operatorCode</code> , <code>triggerType='SAGA_INTENT'</code> , <code>triggerIntentCode=<from envelope></code> , <code>status='ACTIVE'</code>). The query uses the <code>idx_billable_event_trigger_intent</code> partial index.
R-BIL-01-CR-2	For each candidate, BIL-01 evaluates the candidate's <code>trigger_filter_drl</code> rule (if non-null) against the <code>intentContext</code> facts. The rule votes accept/reject. NULL filter = always accept.
R-BIL-01-CR-3	Eligibility filters (<code>eligibility_franchise_refs</code> , <code>eligibility_package_refs</code>) are evaluated post-filter-rule. Subscription's HomePass franchise + Package ref are read from the intent context (or fetched if absent). Empty/NULL = no restriction.
R-BIL-01-CR-4	Applicability is enforced: PREPAID_ONLY rejects subscriptions whose <code>billingMode = POSTPAID</code> ; vice versa.
R-BIL-01-CR-5	Surviving candidates are ordered by (<code>display_order ASC</code> , <code>code ASC</code>) for deterministic dispatch.

3.2 Amount computation

Rule	Description
R-BIL-01-AC-1	Each matched <code>BillableEvent</code> has its amount computed by a pricing formula. The formula is selected from a registry keyed by (<code>billableEventCode</code> , <code>operatorCode</code>). KE seed formulas: <code>prorateDelta</code> , <code>migrationFee</code> , <code>relocationFee</code> , <code>etfResidential</code> , <code>firstCycleCharge</code> , <code>oldCycleCredit</code> , etc. New formulas register via the same registry.
R-BIL-01-AC-2	Pricing formulas receive the intent context + subscription snapshot + cycle window as inputs. They return { <code>amount</code> , <code>sign?</code> , <code>metadata</code> }. For <code>amount_sign_policy = POSITIVE_ONLY</code> events, the formula's sign is forced positive; for <code>NEGATIVE_ONLY</code> , forced negative; for <code>SIGNED</code> , the formula's computed sign is honored.
R-BIL-01-AC-3	Signed proration delta (for <code>MIGRATION_PRORATE_DELTA</code> with <code>amount_sign_policy = SIGNED</code>): formula computes <code>priceDelta = (targetPackagePrice - sourcePackagePrice) × (daysRemaining / cycleLengthDays)</code> . Positive → charge; negative → credit. The sign is reflected in the line item's amount (signed) and the <code>metadata.computedSign</code> .
R-BIL-01-AC-4	<code>cycleAnchorPolicy = PRESERVE</code> : BIL-01 computes the signed proration delta for cycle-days-remaining at the price difference. One line item per matched event (e.g., <code>UPGRADE_PRORATE_DELTA</code> , <code>MIGRATION_PRORATE_DELTA</code> , <code>DOWNGRADE_PRORATE_CREDIT</code> — whichever the catalog match resolves to per filter rules).
R-BIL-01-AC-5	<code>cycleAnchorPolicy = RESET_TO_OPERATION_DATE</code> : BIL-01 returns two line items per matched-event-pair: <code>OLD_CYCLE_CREDIT</code> (NEGATIVE — credits the source-price portion of cycle-days-already-paid) + <code>FIRST_CYCLE_CHARGE</code> (POSITIVE — charges the target Package price for one full cycle starting at the operation date). Filter rules on these events match on <code>cycleAnchorPolicy == RESET_TO_OPERATION_DATE</code> per BIL-CFG-01 v1.0 §2.2 KE seed.
R-BIL-01-AC-6	<code>effectiveTiming = END_OF_CURRENT_CYCLE + cycleAnchorPolicy = PRESERVE</code> : BIL-01 returns zero proration delta (the current cycle finishes on source price/Package; next cycle starts on target). Migration fee + other non-proration fees may still apply.

Rule	Description
R-BIL-01-AC-7	effectiveTiming = SCHEDULED_AT: at request time, BIL-01 computes a *preview* using scheduledCommitAt as the would-be operation date. At actual commit time (when the workflow's timer fires + re-validates), the workflow re-invokes BIL-01 — at which point the commit-time cycle window is used and pricing may differ from the preview. The preview's decisionId is NOT committed to a charge; commit-time invocation creates the authoritative charge.
R-BIL-01-AC-8	Tax is applied per-line-item using each Service's default_tax_group_ref (read from PLM-CFG-01) and PLM-CFG-02's rate. Tax is a separate component of totals (gross, tax, net).

3.3 Aggregation

Rule	Description
R-BIL-01-AG-1	totals.gross = SUM(lineItems[].amount) (signed). totals.tax = SUM(lineItems[].taxAmount) (signed, follows the line amount sign for credits). totals.net = totals.gross + totals.tax. All in the same currency (per R-CR-6).
R-BIL-01-AG-2	If line items span multiple currencies, the call is rejected with MIXED_CURRENCY_LINE_ITEMS (400). All matched events SHOULD share the subscription's currency (enforced upstream by BIL-CFG-01 R-B-4 + the subscription's Package currency).
R-BIL-01-AG-3	payFirstRequired on the response = OR(line.payFirstRequired) across matched events. If any matched event requires pay-first, the workflow waits.

3.4 Settlement

Rule	Description
R-BIL-01-SE-1	PREPAID settlement: BIL-01 calls POST /api/wallet-debit on BIL-01-PAY-01 with {customerId, subscriptionId, amount: totals.net, currency, chargeAttemptId}. Positive net → debit. Negative net → wallet credit (top-up). Zero net → no-op (settlementStatus = NOT_APPLICABLE).
R-BIL-01-SE-2	PREPAID wallet debit may fail with INSUFFICIENT_FUNDS. BIL-01 returns settlementStatus = FAILED with reasonCode = INSUFFICIENT_FUNDS. The workflow's payment-await message catch then fires the operator-configured payment-timeout. Customer-initiated top-up via portal / USSD / agent triggers the payment-confirmed message back to the workflow when the wallet rises to sufficient.
R-BIL-01-SE-3	POSTPAID settlement: positive totals.net → BIL-01 calls POST /api/invoice-lines on BIL-02-GEN-01 with {customerId, subscriptionId, chargeAttemptId, lines: [...], currency, dueDate, originIntent}. Negative totals.net (net credit) is NOT settled here — the call is rejected with settlementStatus = REJECTED, reasonCode = REQUIRES_CREDIT_NOTE_WORKFLOW, and the caller is directed to use BIL-02-ADJ-01 (credit note flow). For POSTPAID, BIL-01 does NOT check credit limit at charge time — that's BIL-04 Collections's job at cycle close.
R-BIL-01-SE-4	A single intent call settles as a single transaction per billing mode: PREPAID → one wallet debit of net total; POSTPAID → one batch invoice-lines call attached to the same invoice. Partial settlement is not supported. If settlement fails, the entire decision rolls back; the workflow receives settlementStatus = FAILED.
R-BIL-01-SE-5	The decisionId is recorded in bil01_charge_ledger (one row per decision) with status PENDING at issue, transitioning to SETTLED / FAILED / REJECTED upon outcome. Idempotency: same (subscriptionId, fulfillmentCorrelationId) retried within 24h returns the prior decision row with HTTP 200.
R-BIL-01-SE-6	When the canonical envelope contains multiple BillableEvents (e.g., MIGRATION_INTENT yielding migration fee + signed proration delta), aggregation runs across all events; settlement is a single transaction per the rules above.

3.5 State callbacks

Rule	Description
R-BIL-01-SC-1	After successful settlement, BIL-01 invokes each matched BillableEvent's <code>state_callback</code> (if non-null) via SUB-LM-01's transition endpoint. Per-event basis — multiple events may each invoke their own callback. Order: same as line item order (<code>display_order</code> , <code>code</code>).
R-BIL-01-SC-2	Callback invocation result is recorded in the response's <code>stateCallbackInvocations[]</code> array — one entry per invocation with <code>transitionCode</code> , <code>outcome</code> (SUCCEEDED / FAILED), <code>failureReason</code> .
R-BIL-01-SC-3	Callback failure does NOT roll back the charge. The charge is recorded as SETTLED; the callback failure is recorded; <code>BillableEventStateCallbackFailed</code> Kafka event fires for alerting. Per BIL-CFG-01 v1.0 R-SC-5.
R-BIL-01-SC-4	For SAGA_INTENT calls where the WORKFLOW itself drives the subscription transition (e.g., UPGRADE-01 commits <code>package_ref</code> change via SUB-LM-01 directly, not via callback), the BillableEvent has <code>state_callback = null</code> . State callbacks are used for events where BIL-01 is the trigger of the transition (e.g., RECONNECTION_FEE_AFTER_DUNNING calls back to flip SUSPENDED_NP → ACTIVE; EQUIPMENT_BUYBACK_ONT calls back to unbind equipment).

3.6 Operator scoping

Rule	Description
R-BIL-01-OP-1	Every envelope MUST carry <code>operatorCode</code> . Catalog resolution is scoped to the operator. Cross-operator catalog matches are not possible.
R-BIL-01-OP-2	Filter rules live in operator-scoped KJAR packages (<code>billing-rules-<operator></code>). The KJAR is selected by <code>operatorCode</code> at decision time. Cross-operator rule references are not possible.
R-BIL-01-OP-3	Pricing formulas are registered per (<code>billableEventCode</code> , <code>operatorCode</code>). Same <code>billableEventCode</code> may have different formulas across operators (e.g., RELOCATION_FEE_CROSS_FRANCHISE may use different distance-based formula in WUG vs WIK).

4. REST API

4.1 POST /api/billing/decide — the bil01-emit-intent worker endpoint

Request and response per §2.1 and §2.2 above.

4.2 POST /api/billing/charge-direct — direct admin/customer charges

For `trigger_type = ADMIN_ACTION / CUSTOMER_PURCHASE / EXTERNAL_PAYMENT`. Caller provides the `BillableEvent` code directly rather than an intent.

```
POST /api/billing/charge-direct HTTP/1.1
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "subscriptionId":    "sub_01HZ_KAMAU_FIBER",
  "customerId":        "cust_01HZ_KAMAU",
  "billableEventCode": "PPV_MOVIE",
  "purchaseContext":   {"movieId": "movie_12345", "duration_hours": 48},
  "fulfillmentCorrelationId": "ppv-purchase-01HZ-KAMAU-12345"
}

→ 200 OK
{
  "decisionId": "bil_dec_01HZ_KAMAU_PPV",
  "lineItems": [{"billableEventCode": "PPV_MOVIE", "amount": 350.00, "currency": "KES"}],
  "totals":     {"gross": 350.00, "tax": 56.00, "net": 406.00, "currency": "KES"},
  "settlementStatus": "SETTLED"
}
```

4.3 GET /api/billing/decisions/{decisionId} — query a prior decision

```
GET /api/billing/decisions/bil_dec_01HZ_OTIENO_MIGRATE HTTP/1.1

→ 200 OK
{full decision envelope as above}
```


4.4 Errors

```
→ 200 OK with settlementStatus = REJECTED
{
  "decisionId":      "bil_dec_01HZ_...",
  "settlementStatus": "REJECTED",
  "reasonCode":      "REQUIRES_CREDIT_NOTE_WORKFLOW",
  "guidance":         "Net total is negative for POSTPAID subscription. Use BIL-02-ADJ-01 credit note flow."
}

→ 200 OK with settlementStatus = FAILED
{
  "decisionId":      "bil_dec_01HZ_...",
  "settlementStatus": "FAILED",
  "reasonCode":      "INSUFFICIENT_FUNDS",
  "walletBalance":    1250.00,
  "amountRequired":   7540.00,
  "guidance":         "PREPAID wallet has insufficient balance. Customer must top up."
}

→ 400 Bad Request
{
  "code":             "NO_MATCHING_BILLABLE_EVENTS",
  "guidance":         "No active BillableEvents in catalog match (operatorCode=WIK, triggerIntentCode=MIGRATION_INTENT, filters)."
}
```

5. Kafka events

Event	Topic	Fires on
BillableEventCharged	sophix.billing.bil01.charged	Settlement succeeds (PREPAID debit confirmed, or POSTPAID invoice line attached). Carries full decision envelope + settlement metadata.
BillableEventChargeFailed	sophix.billing.bil01.chargefailed	Settlement fails (INSUFFICIENT_FUNDS, REQUIRES_CREDIT_NOTE_WORKFLOW, etc.).
BillableEventStateCallbackFailed	sophix.billing.bil01.statecallbackfailed	Successful charge but callback to SUB-LM-01 failed. Alert consumer.

Sample BillableEventCharged:

```
{
  "eventType":      "BillableEventCharged",
  "aggregateId":    "bil_dec_01HZ_OTIENO_MIGRATE",
  "timestamp":      "2026-06-01T09:00:50+03:00",
  "payload": {
    "operatorCode":      "WIK",
    "subscriptionId":    "sub_01HZ_OTIENO_FIBER",
    "customerId":        "cust_01HZ_OTIENO",
    "decisionId":        "bil_dec_01HZ_OTIENO_MIGRATE",
    "intentCode":        "MIGRATION_INTENT",
    "billingMode":       "POSTPAID",
    "lineItems": [
      {"billableEventCode": "MIGRATION_FEE", "amount": 1500.00, "currency": "KES"},
      {"billableEventCode": "MIGRATION_PRORATE_DELTA", "amount": 5000.00, "currency": "KES"}
    ],
    "totals":             {"gross": 6500.00, "tax": 1040.00, "net": 7540.00, "currency": "KES"},
    "settlementStatus":   "SETTLED",
    "invoiceId":          "inv_01HZ_OTIENO_JUN_2026",
    "fulfillmentCorrelationId": "subop_01HZ_OTIENO_MIGRATE: bil01",
    "settledAt":          "2026-06-01T09:00:50+03:00"
  }
}
```

6. Cross-DD coordination

Module	Direction	Contract
BIL-CFG-01 v1.0	Reads catalog	BillableEvent rows + filter rule references
PLM-CFG-01 v1.0	Reads Services	ONE_SHOT Service definitions referenced by BillableEvents; tax-group + wallet refs
PLM-CFG-02 v1.0	Reads tax rates	Tax computation per line item
PLM-CFG-03 v1.0	Reads wallet config	Wallet applicability filter

Module	Direction	Contract
SIP-01	Reads Packages	Source/target Package prices for proration formulas
SUB-WF-*	Caller (synchronous via bil01-emit-intent)	Workflow DDs send intent + context, await response
SUB-LM-01	State callback target	BIL-01 invokes registered transition codes post-settlement
BIL-01-PAY-01	Settlement consumer	PREPAID wallet debits
BIL-02-GEN-01	Settlement consumer	POSTPAID invoice line attachment
BIL-02-ADJ-01	Sibling	Credit note flow for negative POSTPAID totals (BIL-01 redirects there)
BIL-04	Sibling	Collections / dunning — credit limit enforcement at cycle close

6.1 Build order

This DD deploys after:

1. BIL-CFG-01 v1.0
2. PLM-CFG-01 v1.0
3. PLM-CFG-02 v1.0, PLM-CFG-03 v1.0
4. SIP-01 v1.0
5. SUB-LM-01 v1.0
6. BIL-01-PAY-01 v1.0
7. BIL-02-GEN-01 v1.0

Consumers depending on v1.1:

- All SUB-WF-* workflow DDs (synchronous bil01-emit-intent calls)
- BIL-CFG-01 admin tooling (the catalog is meaningless without BIL-01 to read it)

6.2 Stub debt + open coordination items

- **Per-operator pricing formula registries** — KE WIK seed formulas listed in §3.2 R-AC-1; WUG/WTZ/YASSN register their own formulas (or reuse KE's) per operator deep-dives.
- **Per-operator billing-rules KJAR packages** — billing-rules-<operator> packages contain operator-specific filter rules. WUG/WTZ/YASSN rules pending per-operator deep dives.
- callerWorkflowKind **enumeration** — currently free-text in the envelope; pending coordination with workflow DDs to formalize as enum (SUB_WF_ACTIVATE / SUB_WF_PAUSE / SUB_WF_UPGRADE / etc.).

B — Current TZ

Status: To be filled in after codebase cartography review.

A charging engine likely exists in the TZ codebase (the v0.9 baseline references workshop observation). Specific structural details — whether the canonical envelope shape matches, whether signed-amount semantics are present, whether cycle-anchor RESET is handled, whether operator scoping exists — require source-code review to document accurately.